you have to process HTML, which is really the entire point of this section. I'm showing you the correct way to work on existing HTML by using a DOM parser, which will allow you to search and ultimately modify a DOM. We will get to the modification and output functionality of `Dom\HTMLDocument` in due course.

As we saw, the `Dom\XPath::query()` method takes an XPath expression string as an argument, which it uses to find and return matching nodes in the DOM. XPath expressions work a little like filesystem paths: you address a location in the filesystem by specifying a path such as `./documents/git/my_project` (which is an instruction to navigate from the current directory, descend to `documents`, from there to `git`, and so on) with XPath you address the tree structure of the document. Given an XML document like this…

```
<root>
  <documents>
    <git>
      <my_project>
      </my_project>
    </git>
  </documents>
</root>
```

… the XPath expression to address the `<my_project>` node is simply `/root/documents/git/my_project`. Notice how the / character acts just like a directory separator in a file path. In XPath syntax this is a path separator between localization steps: in plain language the tokens on the right of the / are node names that are subordinate to (descendent from) those on the right, which are superordinate (ancestors of). Thus our `root` node is the immediate superordinate/ancestor of `documents`, which in turn is the immediate superordinate/ancestor of `git`… and so on and so on.

The expression `/root/documents` localizes the path to the `documents` node; each node-name expression is a *localization step*, with / being the *path separator*. Localization steps themselves have three parts:

- *Axis*: required, determines navigation direction
- *Node test*: required, filters the node set of the current axis by name or type
- *Predicates*: optional, additional filtering of the node set